

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 30 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Kurangi Arun Kumar /192372287 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

Answer :

1. Simple Reflex Agent

- **How it functions:** Operates entirely on direct, condition-action rules (`IF \text{ condition } THEN \text{ action }`) based strictly on the current percept. It has no memory of past states.
- **Smart Home Examples:**
 - `IF$ Motion Sensor == Active $THEN$ Turn on Hallway Light.`
 - `IF$ Temperature > 26°C $THEN$ Turn on AC.`
 - `IF$ Smoke Detector == Active $THEN$ Sound Alarm.`
- **Limitations:** It fails whenever the environment requires context or history. For instance, if someone is sitting still reading a book, the motion sensor stops detecting movement, and a simple reflex agent would abruptly turn off the lights, ruining comfort.

2. Model-Based Reflex Agent

- **How it functions:** Maintains an internal state (a "model" of the world) to track how the environment evolves and how its own actions affect the world over time. It can handle partial observability.
- **Smart Home Examples:**
 - **Lighting:** Keeps track that a room is occupied even when motion stops momentarily, keeping lights on until no motion is detected for 10 minutes.
 - **Climate:** Remembers that the AC was turned on 5 minutes ago and needs time to cool the room down, preventing it from constantly toggles the unit on and off.
 - **Security:** Tracks door states—if a door opens *after* the security system was armed, it knows an intrusion is happening versus someone just walking around inside.
- **Limitations:** It knows *what the world is doing*, but it doesn't plan ahead for future outcomes or adapt if user preferences change.

3. Goal-Based Agent

- **How it functions:** Combines internal state tracking with explicit **goal information**. It evaluates potential future states to choose actions that lead toward its set goals.
- **Smart Home Examples:**
 - **Goal:** "Maintain indoor climate at 22°C by 6:00 PM."

- **Behavior:** The agent looks at current time, outside temperature, and historical cooling rates to start pre-cooling the house at 5:15 PM so the goal is met right as the user gets home.
- **Limitations:** Goals are usually binary (either achieved or not). If there are multiple ways to reach a goal or conflicting goals (e.g., maintaining 22°C vs. saving electricity), it doesn't know how to choose the *best* trade-off.

4. Utility-Based Agent

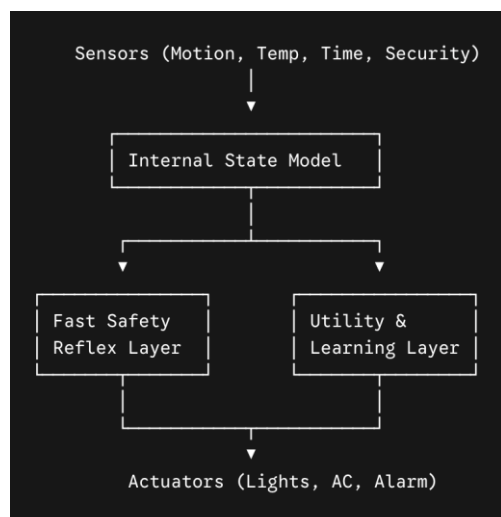
- **How it functions:** Uses a **utility function** that maps a state (or sequence of states) to a real number measuring "happiness," "comfort," or "efficiency." It handles trade-offs when multiple goals conflict.
- **Smart Home Examples:**
 - **Trade-off Management:** If electricity rates spike during peak afternoon hours, the utility-based agent balances two competing factors: $Utility = w_1(\text{Comfort}) - w_2(\text{Energy Cost})$.
 - **Action:** It might allow the house to warm up by 1.5°C during high-tariff hours to save money, then cool it down later, maximizing total user utility.

Comparison of Agent Types in Smart Home Control

Agent Type	Real-Time Response	Handles Memory/History	Balances Conflicts	Learns & Adapts
Simple Reflex	Fast	No	No	No
Model-Based	Fast	Yes	No	No
Goal-Based	Moderate	Yes	Basic	No
Utility-Based	Moderate to High	Yes	Excellent	Yes (when paired with Learning)

Best Approach: Hybrid Utility-Based Learning Agent

A purely single-agent architecture falls short for a full smart home. The most effective design is a **Hybrid Utility-Based Learning Agent**.



Justification:

1. **Safety & Speed (Reflex Layer):** Emergency security events (smoke detected, intruder breach) need instant responses ($<10\text{ms}$). These bypass complex utility calculations and run on fast reflex/model-based rules.
2. **Comfort vs. Energy Trade-offs (Utility Engine):** Normal routines (heating, cooling, ambient lighting) involve trade-offs. A utility function optimizes user comfort against utility bill costs dynamically based on peak tariff rates.
3. **User Preference Adaptability (Learning Element):** Over time, a learning component (like Q-learning or neural networks) monitors manual overrides (e.g., the user manually lowering the temperature every Friday night) and updates the utility function weights to adapt without manual programming.

This hybrid structure provides the safety of reflex rules, the contextual awareness of model-based tracking, and the smart trade-off management of utility learning.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Answer :

Uninformed Search Strategies in Unknown Mazes

When a robot is placed in an unknown maze, it cannot use heuristic-based search strategies like A* because it lacks spatial coordinates or straight-line distance estimates to the exit. It must rely on uninformed search strategies using only local sensor inputs (e.g., detecting wall obstructions in adjacent cells).

- **Uniform Cost Search (UCS):**

- **Function:** Explores nodes in order of increasing path cost $g(n)$. If movement costs vary across terrain (e.g., smooth tiles vs. rough gravel), UCS guarantees finding the lowest-cost path to the exit.
- **Behavior in Unknown Mazes:** As the robot discovers neighboring cells, it updates path costs. UCS prioritizes exploring cheaper paths before attempting higher-cost movements.

- **Iterative Deepening Search (IDS):**

- **Function:** Combines the depth-first search (DFS) space efficiency with the breadth-first search (BFS) completeness and optimality (for uniform step costs). It performs repeated depth-limited searches, increasing the depth limit L by 1 in each iteration ($L = 0, 1, 2, \dots$).
- **Behavior in Unknown Mazes:** IDS explores all paths up to depth L . If the exit is not found, the robot resets or backtracks and searches up to depth $L+1$, systematically discovering the shortest-step route.

Complexity Analysis

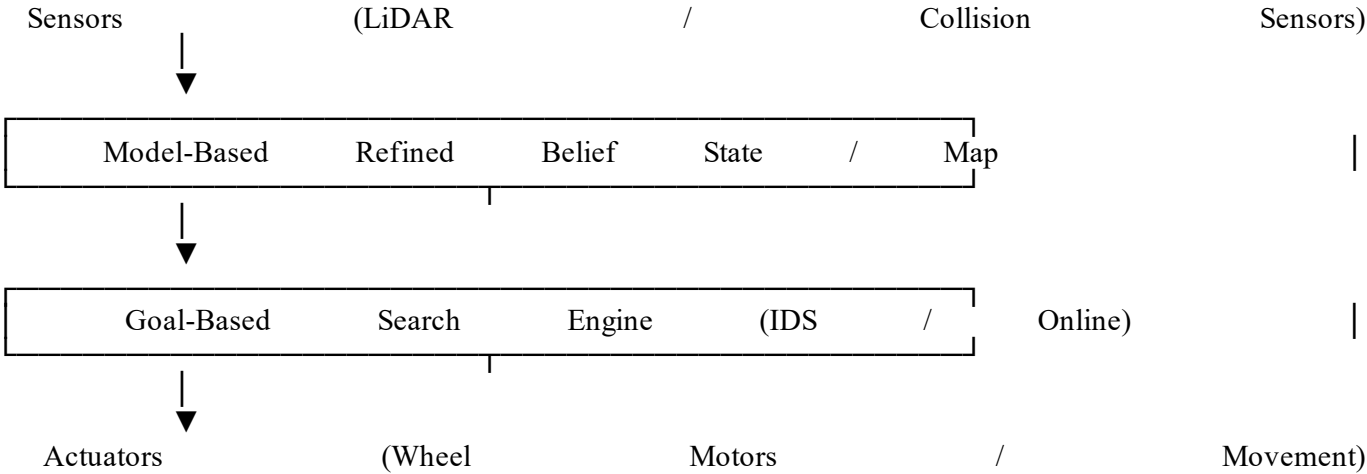
Let b be the branching factor (typically $b \leq 3$ in a 2D grid maze) and d be the depth of the optimal solution. Let C^* be the cost of the optimal solution and ϵ be the minimum step cost.

Algorithm	Time Complexity	Space Complexity	Optimality	Completeness
Uniform Cost Search (UCS)	$O(b^{\lceil 1 + \lceil C^* / \epsilon \rceil \rceil})$	$O(b^{\lceil 1 + \lceil C^* / \epsilon \rceil \rceil})$	Yes (for $c(a) \geq \epsilon > 0$)	Yes
Iterative Deepening Search (IDS)	$O(b^d)$	$O(b \cdot d)$	Yes (for uniform step costs)	Yes

- UCS Trade-offs:**
 - Advantage:* Guaranteed to find the optimal path even when action costs vary.
 - Disadvantage:* Extremely memory-intensive. It keeps all generated nodes in memory (in the priority queue), which quickly leads to memory exhaustion in large mazes.
- IDS Trade-offs:**
 - Advantage:* Highly space-efficient ($O(b \cdot d)$ memory), making it ideal for physical hardware with memory constraints.
 - Disadvantage:* Re-evaluates upper-level nodes multiple times across iterations, leading to redundant local explorations if physical backtracking is expensive.

Agent Types and Impact on Decision-Making

How the robot navigates the maze depends on its internal architectural design:



- Simple Reflex Agent:**
 - Mechanism:* Uses direct condition-action rules (e.g., $\text{IF } \text{wall ahead} \text{ THEN } \text{turn right}$).
 - Impact:* Fails in complex mazes. It easily gets trapped in infinite loops or dead ends because it has no memory of previously visited cells.
- Model-Based Agent:**
 - Mechanism:* Maintains an internal map (occupancy grid) of explored cells and visited paths.
 - Impact:* Avoids re-exploring dead ends and loops, allowing online mapping as sensors reveal new walls.
- Goal-Based Agent:**
 - Mechanism:* Combines the internal model with an explicit goal state (Exit Cell).
 - Impact:* Uses search algorithms to plan sequences of actions toward the exit using its built-in model.

4. Utility-Based Agent:

- *Mechanism:* Uses a utility function to evaluate trade-offs between reaching the exit, minimizing energy consumption, and avoiding risky terrain.

Recommended Combination & Justification

Recommendation: Model-Based Goal Agent paired with Online Iterative Deepening Search (or Online DFS / Pledge Algorithm with Mapping).

- **Justification:**
 - **Memory Limits on Physical Hardware:** Physical robots have limited RAM. UCS requires storing the entire frontier in memory, which scales exponentially with maze size. IDS keeps memory requirements linear ($O(b \cdot d)$).
 - **Avoiding Loops (Model-Based Component):** A pure reflex agent cannot solve arbitrary mazes. The model-based aspect allows the robot to store its local occupancy grid in real time, preventing cyclic loops and ensuring the agent knows which directions remain unexplored at any junction.
 - **Step Uniformity:** If maze steps have uniform energy costs, IDS delivers optimal path length while operating within strict hardware memory bounds.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand